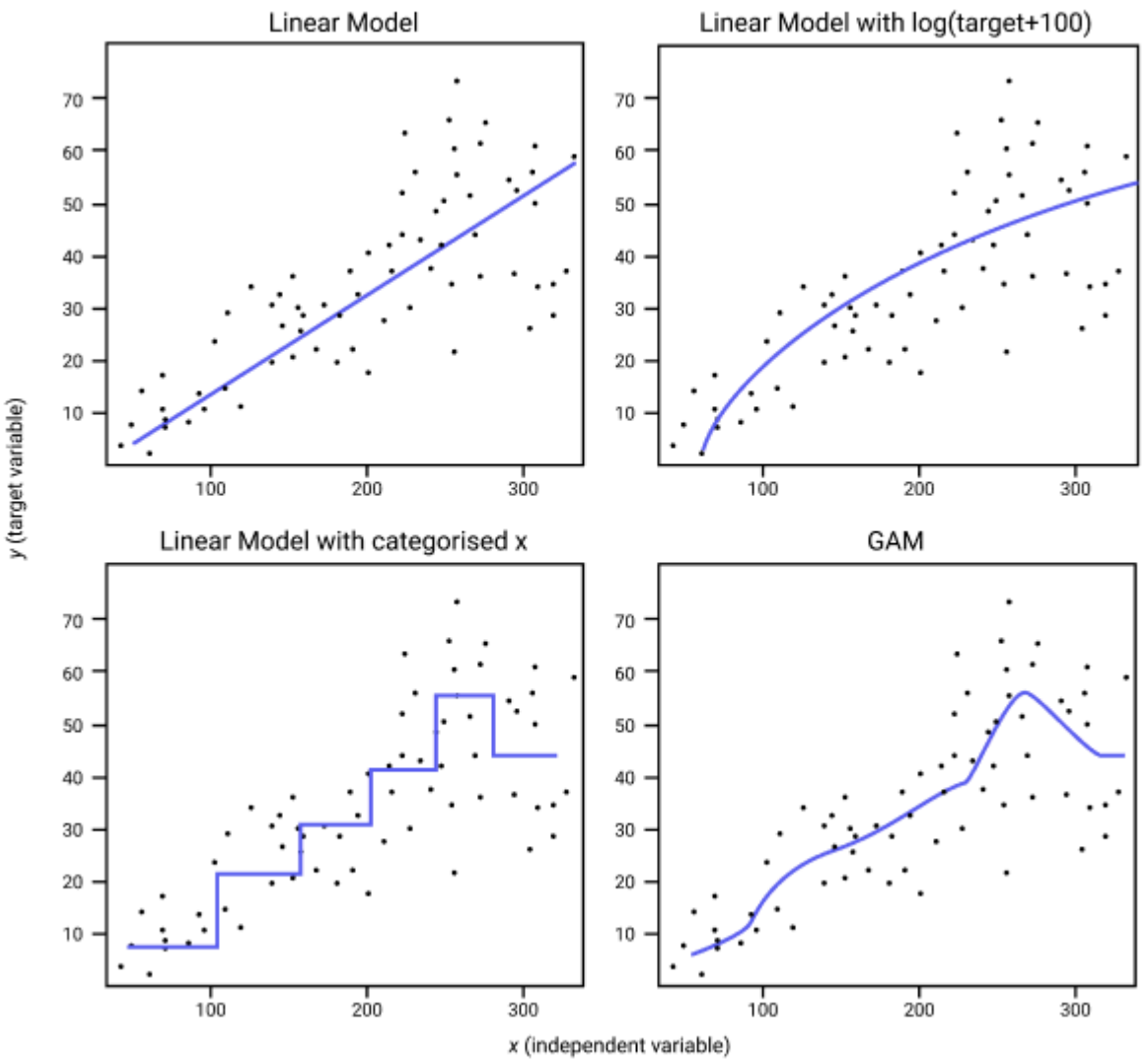
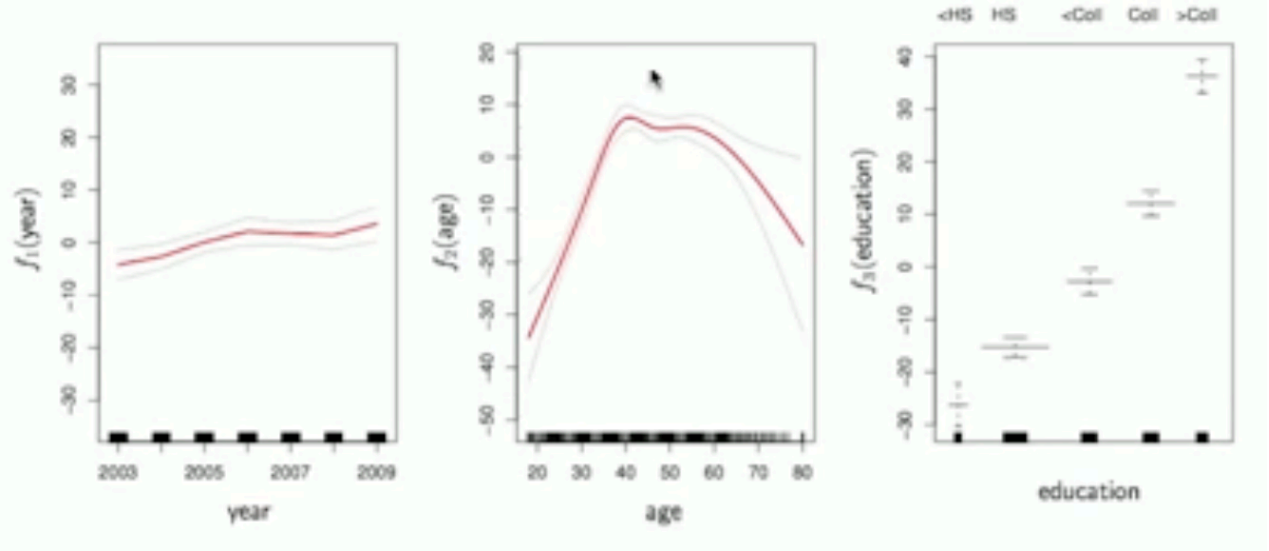
GAMs = Generalised Aditive Models

Idea: Extend multiple linear regression to allow **non-linear** relationships for **each predictor**, but keep them **additive** (no interactions by default).



- Model: we simply replace the beta (coefficient) with the flexible function that allow non-linear function

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i.$$



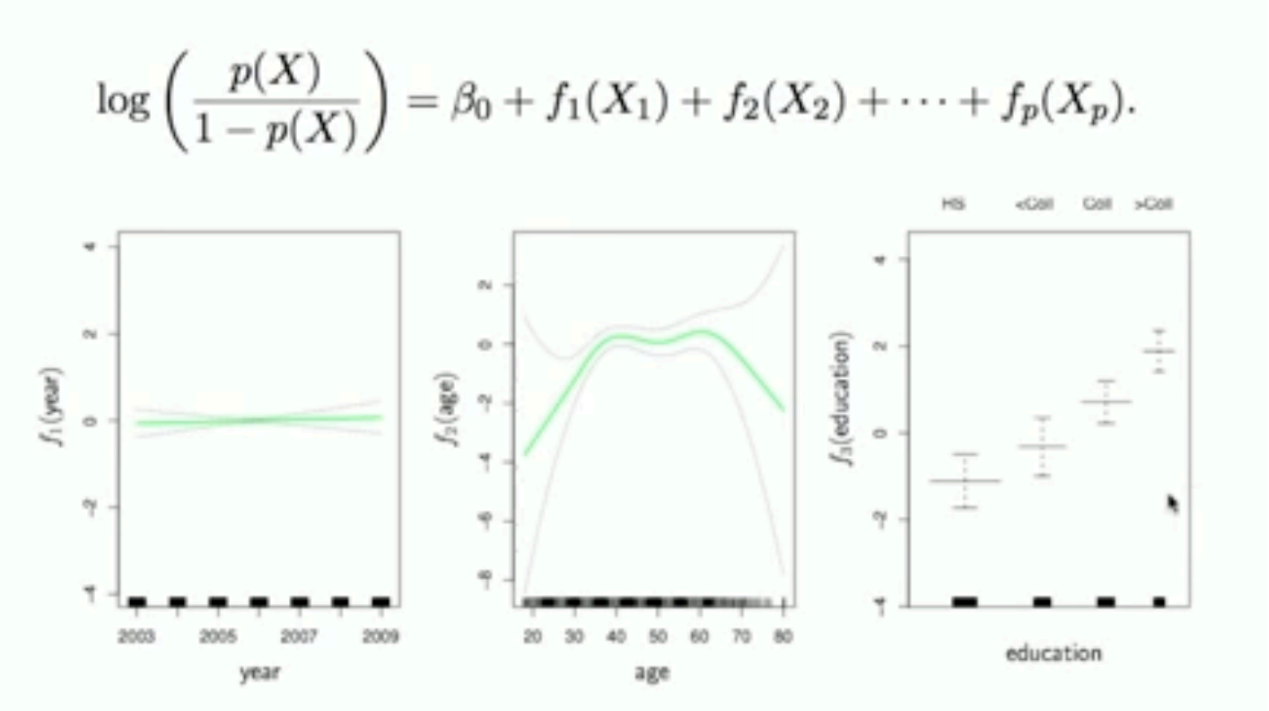
(g is the link function, like in GLMs)

- Each f_j(Xin) is a smooth function (can be smoothing spline, regression spline, polynomial, local regression, etc.).
- Can fit a GAM simply using, e.g. natural spline

$$\texttt{lm(wage} \sim \texttt{ns(year, df} = 5) + \texttt{ns(age, df} = 5) + \texttt{education)}$$

- Coefficient not that interesting; fitted function are. the previous plot was produced using plot.gam
- can mix terms, some linear, some nonlinear, and use **anova to compare model**
- **For regression problems** (continuous Y): usually Gaussian family --> ordinary additive model.
- **For classification** (e.g., logistic GAM): use logistic link → models log-odds additively.

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p).$$



- **Big advantages**:
  - Much more flexible than linear models
  - Still interpretable (you can plot each $f_j(X_j)$ separately to see the effect of that variable)
  - Can capture very different shapes for different predictors
- **Fitting**: Usually via **backfitting** (iteratively smooth each variable holding others fixed) or penalized methods.

```
from pygam import LinearGAM, s

gam = LinearGAM(s(0) + s(1))
gam.fit(X, y)
```

Pros

- Very flexible
- Captures complex nonlinear patterns
- Handles multiple features well
- More interpretable than black-box models
- it control the risk of overfitting via smoothing or spline

Cons

- Slower than polynomial regression
- Requires tuning smoothness
- Needs external libraries (e.g., pygam)

Back Fitting

This method fits a model involving multiple predictors by repeatedly updating the fit for each predictor in turn, holding the others fixed. The beauty of this approach is that each time we update a function, we simply apply the fitting method for that variable to a partial residual.